以蒙地卡羅實驗驗證 J-B 檢定統計量並與比較其他檢定方法比較

學號：411078060

姓名：黃琮茗

作品目標: 這次的作品中，面對多個不同(Q1,Q2,Q3)的統計量時，去檢視它們的檢定力是否為良好，且可以透過圖片分析，數值分析兩個方面去檢視。那麼在接下來，可以透過不同常態檢定用法去比較效果如何在這次的作品中遇到最困難的問題是把在做出不同常態檢定法相比較檢定力的部分。檢視整個過程比起以往都要多跑很多時間，所以可以先將數值都先少量化，確定運行結果沒問題。知道成果完進行分析，可以知道在什麼樣的分配下，用哪一種方法會有檢定力上不同的解釋，那些好，那些不好。

研究一:

令 $\{x\,i, i=1\ldots n\}$ 代表來自標準常態 $N(0,1)$ 的 n 個隨機樣本。

統計量 G_1 表示為，$G_1 = \sqrt{\frac{n}{s}}\,\hat{s}$ 其中 $\hat{s}$為偏態係數（skewness）的估計值（參考指令 scipy.stats.skew)。請利用蒙地卡羅模擬（Monte Carlo Simulation）驗證統計量 G_1 服從標準常態 $N(0,1)$。其中蒙地卡羅模擬的環境設定（scenarios）為：

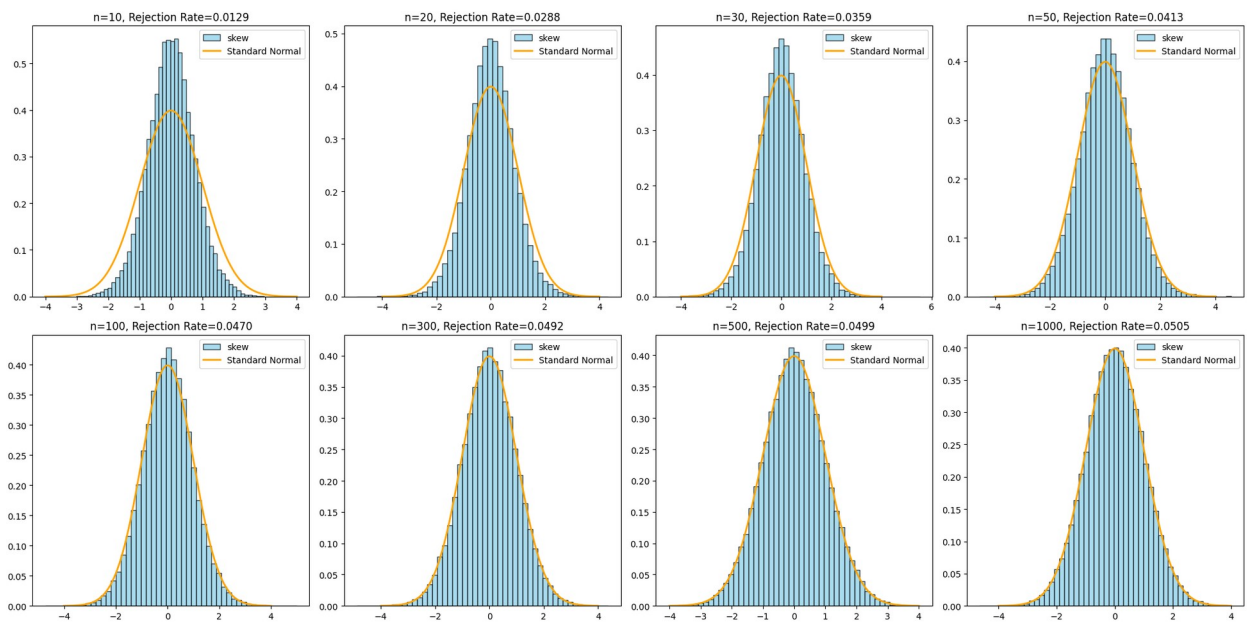
樣本數 $n=10,20,30,50,100,300,500,1000$。

針對每個樣本數 n，模擬次數皆為 $N=50,000$。

繪製 $n=10$ 與 $n=500$ 時，統計量 G_1 的直方圖與 $ECDF$ 圖。並分別畫上對應的標準常態 PDF 與 CDF 圖。

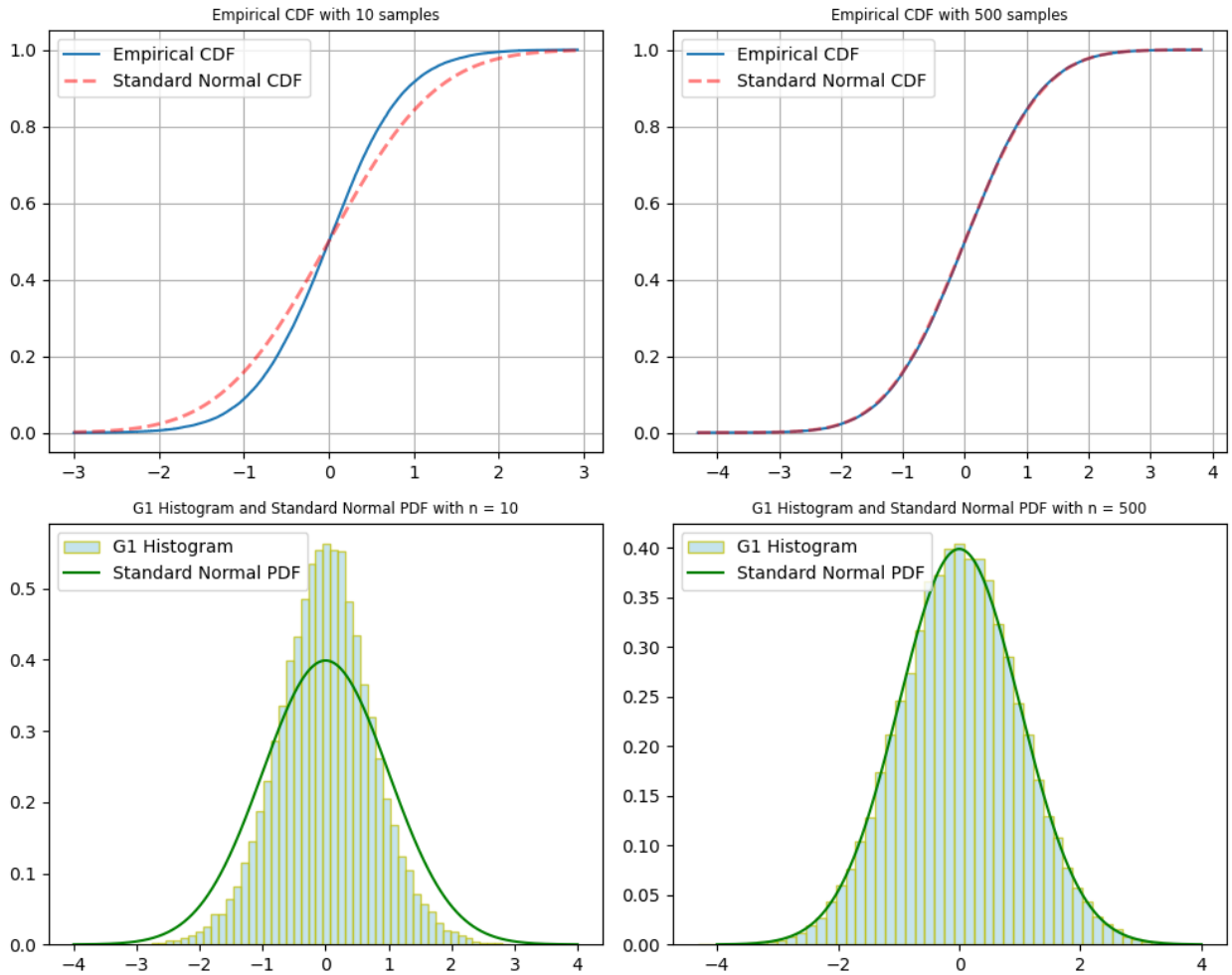
```
from scipy.stats import chi2, norm, kurtosis,skew
import numpy as np
import matplotlib.pyplot as plt

# 設定不同的樣本數
sample_sizes = [10,20,30,50, 100,300,500,1000] # 樣本大小
N = 50000 #抽樣次數
alpha = 0.05 # 顯著水準設定為0.05
CV = norm.ppf(1 - alpha / 2)  # 雙尾區間的臨界值
xx = np.linspace(-4, 4, 200)
fig, ax = plt.subplots(2, 4, figsize=(20,10 ))  # Create a 2x3 grid of
subplots
ax = ax.flatten()  # Flatten the array of axes to make it easier to
iterate over
#
for i, n in enumerate(sample_sizes): #這裡利用enumerate來列舉出
sample_sizes裡面的值i沒有給定所以是從0開始
    x = norm.rvs(0,1, size=(N, n))
```

```
    g1 = np.sqrt(n / 6) *skew(x,axis=1)#利用第一題的公式來計算g1 用到skewness

    ax[i].hist(g1, bins=50, density=True, label='skew', alpha=0.7, color='skyblue', edgecolor='black')  # 畫出g1的直方圖
    ax[i].plot(xx, norm.pdf(xx), label='Standard Normal', color='orange', linewidth=2)  #做出標準常態分配的圖形 用來比較G1是否為常態分配

    reject = np.abs(g1) > CV #只會有0,1
    reject_rate = np.mean(reject)#所以直接取平均就是拒絕率

    ax[i].set_title(f'n={n}, Rejection Rate={reject_rate:.4f}', fontsize=12)  # 調整字體大小
    ax[i].legend(fontsize=10)  # 調整圖例字體大小

plt.tight_layout() # 調整間距
plt.show()
```



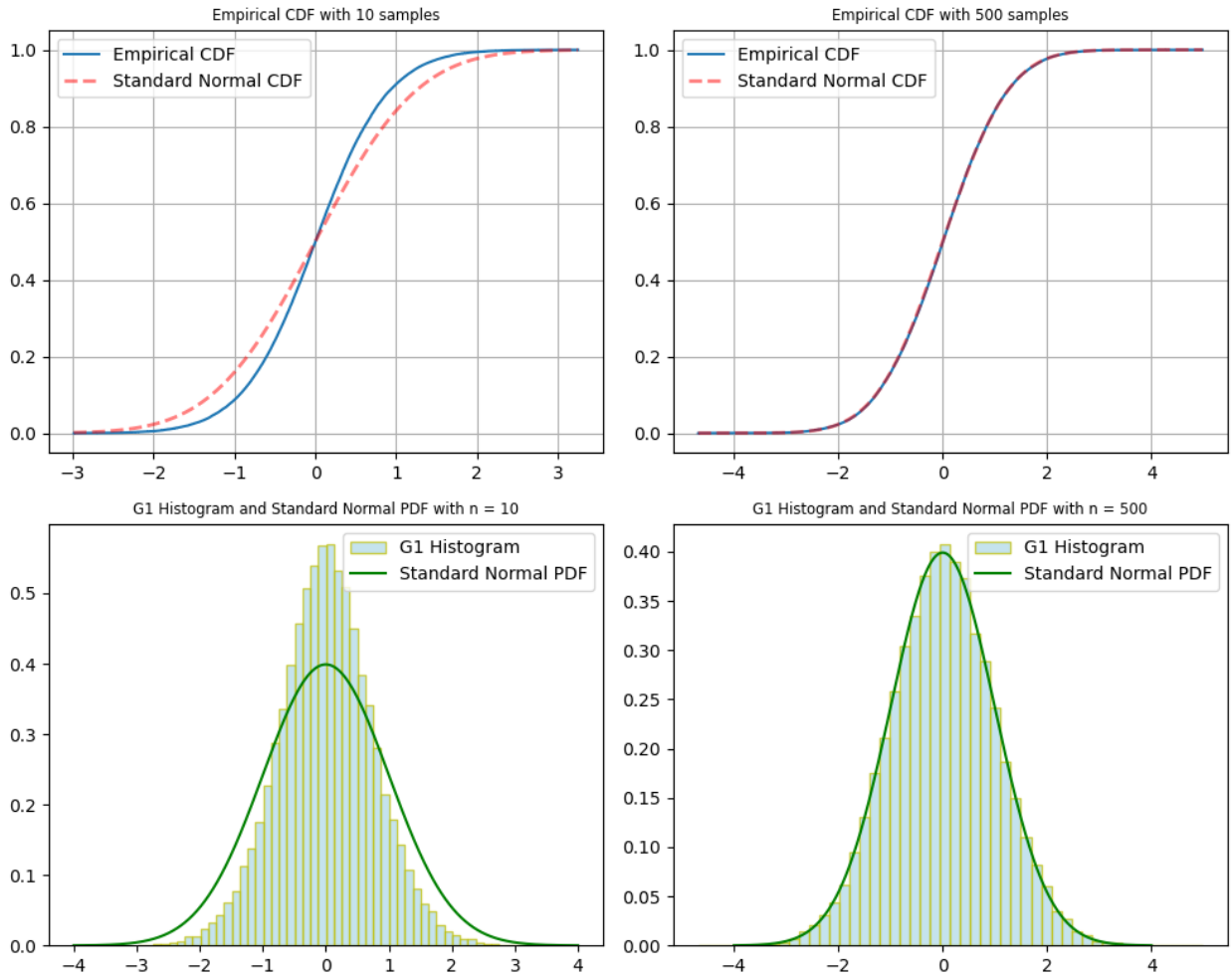

```
from scipy.stats import norm, skew
import numpy as np
import matplotlib.pyplot as plt

# 定義計算G_1 函數
def calculate_G1(xs, n):
    return np.sqrt(n / 6) * skew(xs)

# 定義兩種不同的樣本大小
n_values = [10, 500]  # 樣本大小
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
N = 50000  # 實驗次數
```

```
for i, n in enumerate(n_values):
    g1_values = []  # 創造G_1 值的列表
    for _ in range(N):
        xs = norm.rvs(size=n) # 生成樣本
        g1 = calculate_G1(xs, n) #計算出G1 的值
        g1_values.append(g1)  #將G1 的值加入列表中

    g1_values = np.array(g1_values).flatten()  # 將列表轉換為numpy 陣列並展平 轉化為一維數據

    x_sorted = np.sort(g1_values)#將數據進行排序
    Y = np.arange(1, N + 1) / N
    row = i // 2 #這裡是用來決定圖形要放在哪一列i 帶入0 會得出0//2=0 代表放在第一列
    col = i % 2  #這裡是用來決定圖形要放在哪一行i 帶入0 會得出0%2=0 代表放在第一行

    ax = axs[row, col]#可以利用這種方式來編排圖形位置
    # 繪製ECDF 圖和標準常態分佈的CDF 蘭進行比對
    ax.plot(x_sorted, Y, drawstyle='steps-pre', label='Empirical CDF') # 繪製ECDF 圖
    Y_norm = norm.cdf(x_sorted)
    ax.plot(x_sorted, Y_norm, color='r', linestyle='--', linewidth=2, alpha=0.5, label='Standard Normal CDF')# 標準常態分佈的CDF
    ax.legend()
    ax.grid(True)
    ax.set_title('Empirical CDF with {} samples'.format(n), fontsize='small')

  # 繪製G_1 的值方圖和標準常態分佈的PDF 對比
    ax_hist_pdf = axs[1, col]  # # 直接指定下半部位置
    ax_hist_pdf.hist(g1_values, color='lightblue', edgecolor='y', bins=50, density=True, alpha=0.7, label='G1 Histogram')
     #直方圖再加上edgecolor 使得邊緣變得明顯
    x = np.linspace(-4, 4, 1000)
    pdf = norm.pdf(x)
    ax_hist_pdf.plot(x, pdf, color='g', label='Standard Normal PDF') #標準常態分佈的PDF
    ax_hist_pdf.legend()
    ax_hist_pdf.set_title('G1 Histogram and Standard Normal PDF with n = {}'.format(n), fontsize='small') # 設定標題

plt.tight_layout()
plt.show()
```




```python
from scipy.stats import norm, skew
import numpy as np
import matplotlib.pyplot as plt

# 定義計算G_1 函數
def calculate_G1_vectorized(n):
    xs = norm.rvs(size=(N, n))  # 生成多組樣本，N 組，每組 n 個
    g1 = np.sqrt(n / 6) * skew(xs, axis=1)  # 計算G_1
    return g1

# 定義兩種不同的樣本大小
n_values = [10, 500]  # 樣本大小
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
N = 50000  # 實驗次數

for i, n in enumerate(n_values):
    g1_values = calculate_G1_vectorized(n)  # 計算G_1 的值

    x_sorted = np.sort(g1_values)  # 排序G_1 值
    Y = np.arange(1, N + 1) / N
```

```
    row = i // 2
    col = i % 2

    ax = axs[row, col]
    # 繪製ECDF 圖和標準常態分佈的CDF 進行比對
    ax.plot(x_sorted, Y, drawstyle='steps-pre', label='Empirical CDF')

    Y_norm = norm.cdf(x_sorted)
    ax.plot(x_sorted, Y_norm, color='r', linestyle='--', linewidth=2,
alpha=0.5, label='Standard Normal CDF')
    ax.legend()
    ax.grid(True)
    ax.set_title('Empirical CDF with {} samples'.format(n),
fontsize='small')

    # 繪製G_1 的直方圖和標準常態分佈的PDF 對比
    ax_hist_pdf = axs[1, col]
    ax_hist_pdf.hist(g1_values, color='lightblue', edgecolor='y',
bins=50, density=True, alpha=0.7, label='G1 Histogram')
    x = np.linspace(-4, 4, 1000)
    pdf = norm.pdf(x)
    ax_hist_pdf.plot(x, pdf, color='g', label='Standard Normal PDF')
    ax_hist_pdf.legend()
    ax_hist_pdf.set_title('G1 Histogram and Standard Normal PDF with n
= {}'.format(n), fontsize='small')

plt.tight_layout()
plt.show()
```



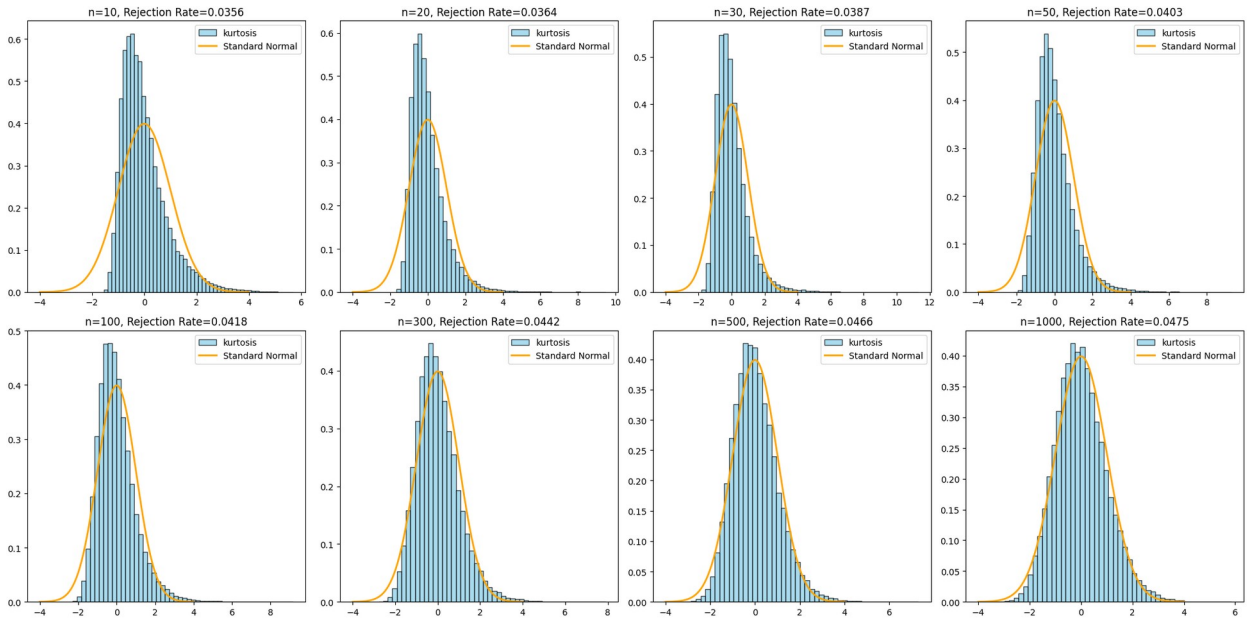
討論注意事項與討論:

- 樣本數在 n=10,30,50 rejection rate 不足，還位於 0.01~0.04 可以從圖片中發現 n=1000 的時候 rejection rate 已經在 0.05 附近浮動了(與常態穩合要在 0.05)
- 從圖形上來看搭配分析，可以發現 n=10 與 n=1000 在常態 N(0,1)上的貼合度是不同的，可以發現樣本數小的時候，中間部分區塊累積到超越常態上分，讓 reject rate 拒絕的量不夠(不足以到達 0.05，也就是常態)，而在樣本數夠大的時候，整體而言貼合度高，且 reject rate 也接近 0.05。
- 在下方 ECDF、CDF 用了迴圈與向量化的作法 迴圈耗時 44 秒 向量耗時 3 秒。
- n=500 ECDF 與實際的 CDF 貼合。
- cdf 圖中可以發現當樣本數越來越大時 ECDF 會越來越貼合 real CDF。

研究二:

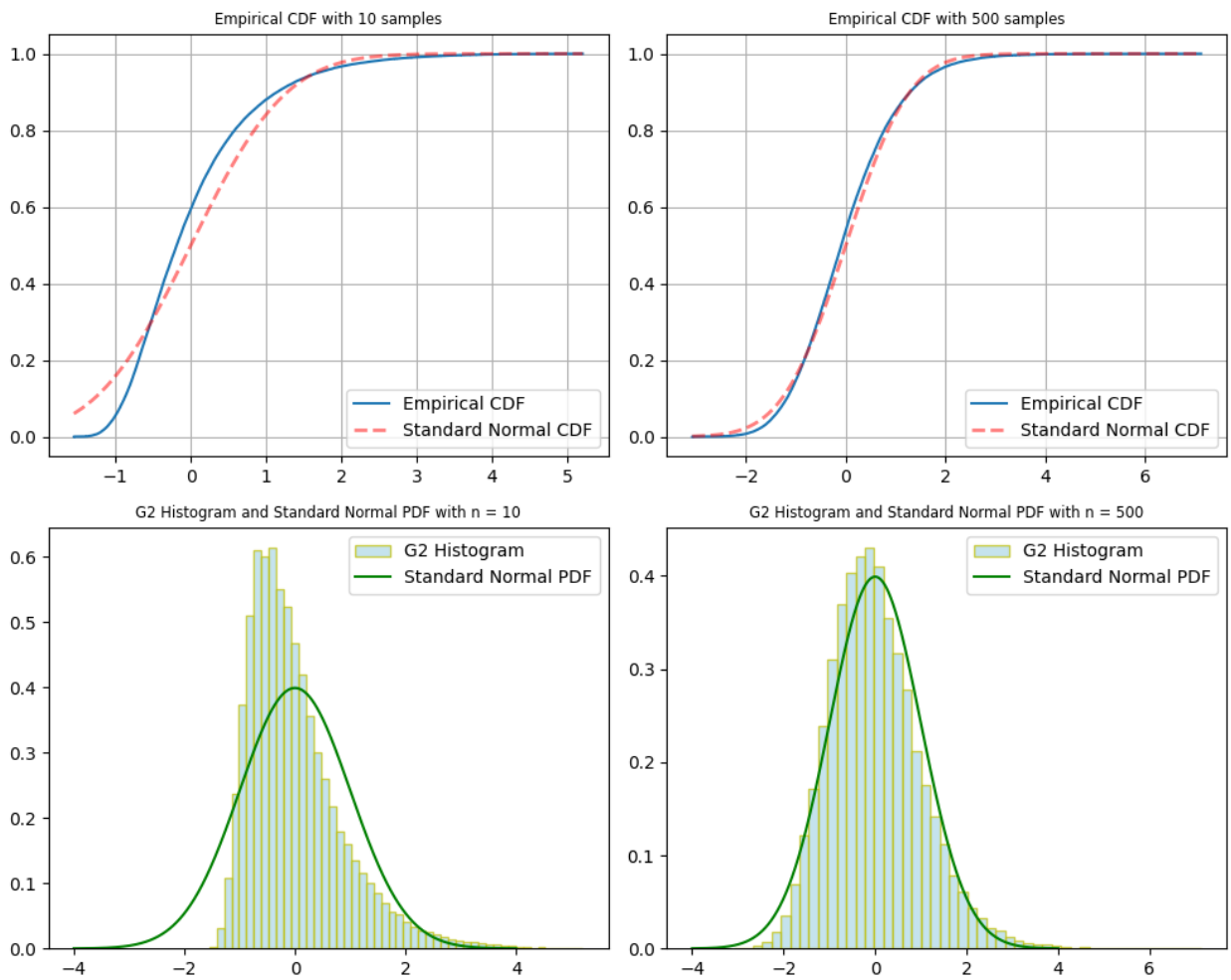
同上，但令統計量為: $G_2 = \sqrt{\frac{n}{24}}\left(\hat{k}-3\right)$ 其中 $\hat{k}$ 為峰態係數（Kurtosis）的估計值（參考指令 scipy.stats.kurtosis）。同樣利用蒙地卡羅模擬，驗證統計量 G_2 服從標準常態 $N(0,1)$。蒙地卡羅模擬的環境設定同上。

```
from scipy.stats import norm, kurtosis
import numpy as np
import matplotlib.pyplot as plt

# 設定不同的樣本數
sample_sizes = [10, 20, 30, 50, 100,300,500, 1000] # 樣本大小
N = 50000 # 抽樣次數
alpha = 0.05 # 顯著水準設定為0.05
CV = norm.ppf(1 - alpha / 2)  # 雙尾區間的臨界值
fig, ax = plt.subplots(2, 4, figsize=(20,10 ))  # Create a 2x3 grid of
subplots
ax = ax.flatten()  # Flatten the array of axes to make it easier to
iterate over
xx = np.linspace(-4, 4, 200)
for i, n in enumerate(sample_sizes):#這裡利用enumerate來列舉出
sample_sizes裡面的值i沒有給定所以是從0開始
    x = norm.rvs(0, 1, size=(N, n))#已知條件設定x為N(0,1)
    g2 = np. sqrt(n/24)*kurtosis(x,bias=False ,axis=1)
    #繪製出直方圖與標準常態分配的圖形進行比較
    ax[i].hist(g2, bins=50, density=True, label='kurtosis', alpha=0.7,
color='skyblue', edgecolor='black')
    ax[i].plot(xx, norm.pdf(xx), label='Standard Normal',
color='orange', linewidth=2)

    reject = np.abs(g2) > CV
    reject_rate = np.mean(reject) #與上面相同

    ax[i].set_title(f'n={n}, Rejection Rate={reject_rate:.4f}',
fontsize=12)
    ax[i].legend(fontsize=10)
plt.tight_layout()
plt.show()
```




```
from scipy.stats import norm, kurtosis
import numpy as np
import matplotlib.pyplot as plt
# 定義計算G_2 函數
def calculate_G2_vectorized(n):
    xs = norm.rvs(size=(N, n))  # 生成多組樣本，N 組，每組 n 個
    g2 = np.sqrt(n / 24) * kurtosis(xs, bias=False, axis=1)  # 計算G_2
    return g2

# 定義兩種不同的樣本大小
n_values = [10, 500]  # 樣本大小
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
N = 50000  # 實驗次數

for i, n in enumerate(n_values):
    g2_values = calculate_G2_vectorized(n)  # 計算G_2 的值

    x_sorted = np.sort(g2_values)  # 排序G_2 值
    Y = np.arange(1, N + 1) / N
    row = i // 2
    col = i % 2

    ax = axs[row, col]
    # 繪製ECDF 圖和標準常態分佈的CDF 進行比對
    ax.plot(x_sorted, Y, drawstyle='steps-pre', label='Empirical CDF')

    Y_norm = norm.cdf(x_sorted)
    ax.plot(x_sorted, Y_norm, color='r', linestyle='--', linewidth=2,
alpha=0.5, label='Standard Normal CDF')
    ax.legend()
    ax.grid(True)
```

```
    ax.set_title('Empirical CDF with {} samples'.format(n),
fontsize='small')

    # 繪製G_2 的直方圖和標準常態分佈的PDF 對比
    ax_hist_pdf = axs[1, col]
    ax_hist_pdf.hist(g2_values, color='lightblue', edgecolor='y',
bins=50, density=True, alpha=0.7, label='G2 Histogram')
    x = np.linspace(-4, 4, 1000)
    pdf = norm.pdf(x)
    ax_hist_pdf.plot(x, pdf, color='g', label='Standard Normal PDF')
    ax_hist_pdf.legend()
    ax_hist_pdf.set_title('G2 Histogram and Standard Normal PDF with n
= {}'.format(n), fontsize='small')

plt.tight_layout()
plt.show()
```



注意事項與討論:

- 樣本數在 n=10,30,50 rejection rate 不足，還位於 0.03~0.04 可以從圖片中發現 n=1000 的時候 rejection rate 已經在 0.05 附近浮動了(與常態穩合要在 0.05)
- 從圖形上來看搭配分析，可以發現 n=10 與 n=1000 在常態 N(0,1)上的貼合度是不同的，可以發現樣本數小的時候，右偏的情形嚴重，讓 reject rate 拒絕的量不夠(不足以到達 0.05，也就是常態)，而在樣本數夠大的時候，整體而言貼合度高且明顯近似常態，且 reject rate 也接近 0.05。
- 從 cdf 圖也可以發現當樣本比較小時曲線的偏移量是更大的，但是樣本數上升 n=(500)就會幾乎貼近常態分佈的曲線。

研究三:

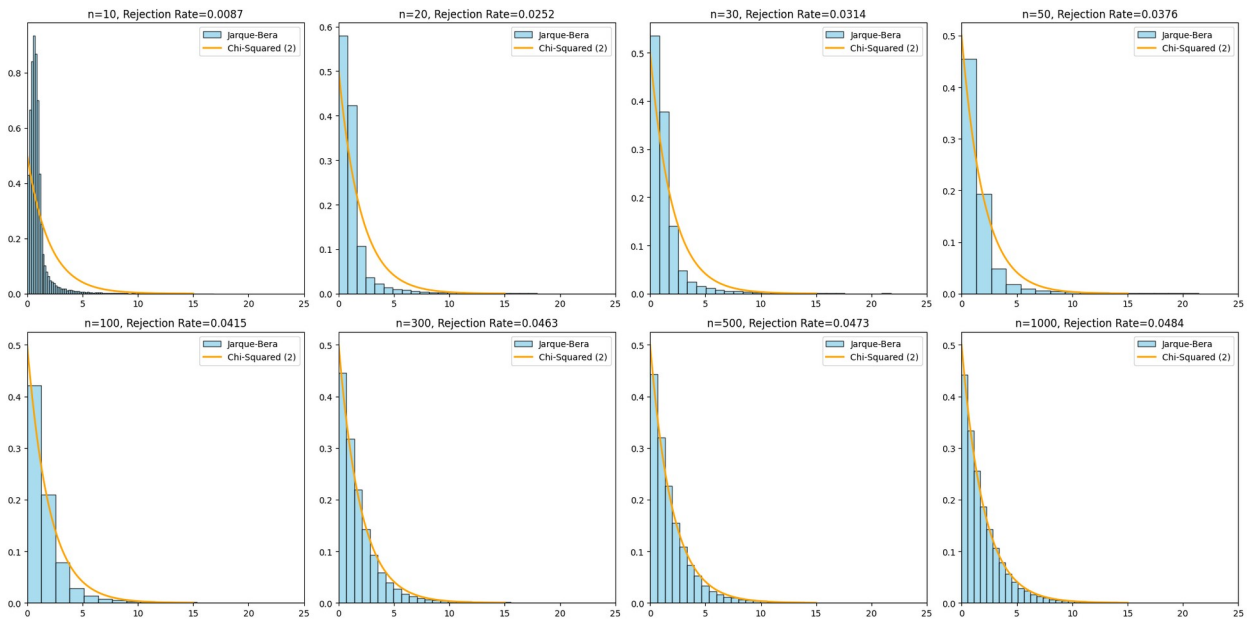
同上，但統計量為

$$G_3 = G_1^2 + \$G_2^2 \$ = \frac{n}{6}\left(\hat{s}^2 + \frac{(\hat{k}-3)^2}{4}\right)$$

同樣利用上述的蒙地卡羅模擬，驗證統計量 G_3 服從卡方分配 $\chi^2(2)$。G_3 為著名的 J-B (Jarque-Bera) 常態檢定統計量。

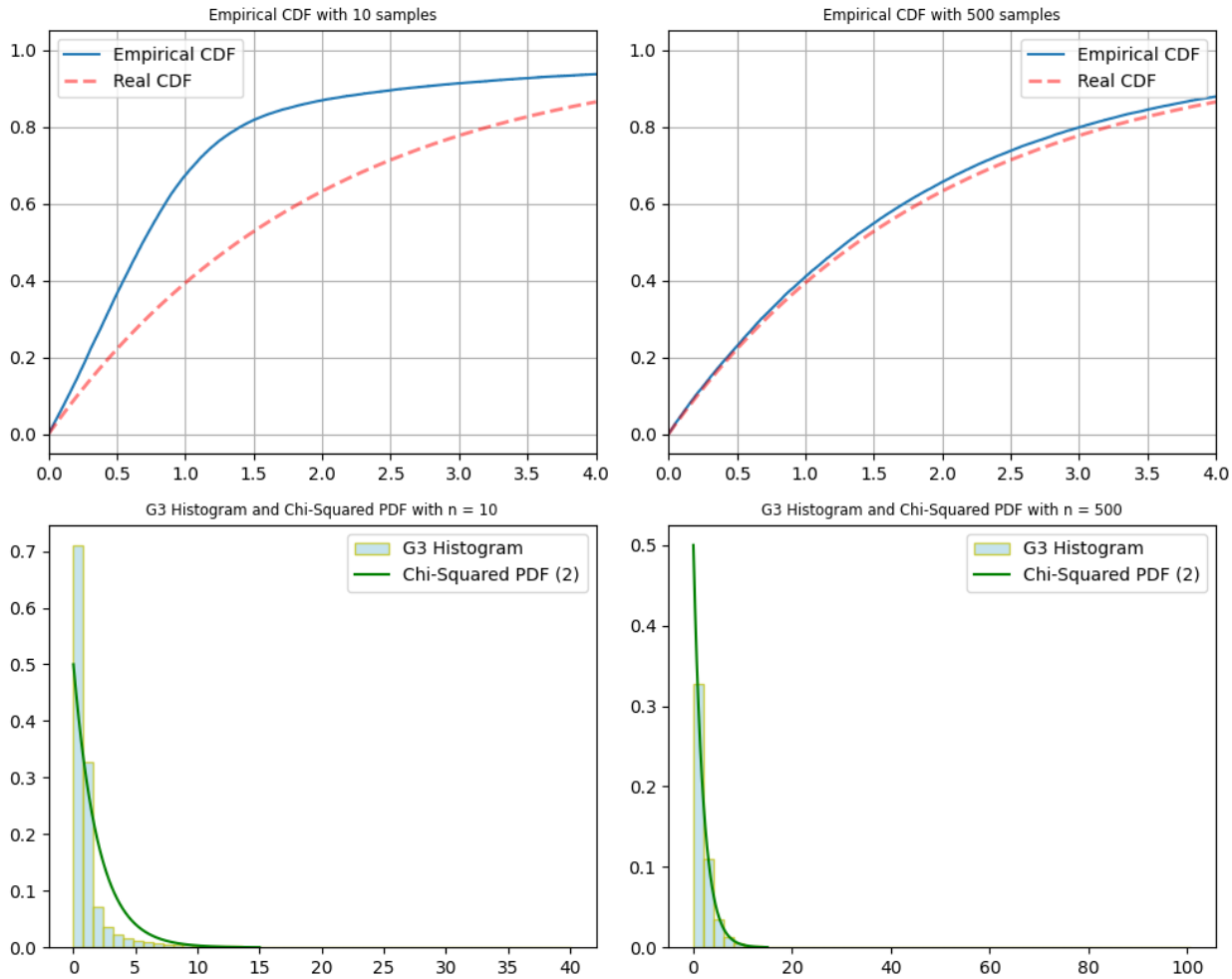
```
from scipy.stats import chi2, norm, kurtosis ,skew
import numpy as np
import matplotlib.pyplot as plt

# 設定不同的樣本數
sample_sizes = [10, 20, 30, 50, 100,300, 500, 1000] # 樣本大小
N = 50000 # 抽樣次數
alpha = 0.05 # 顯著水準設定為0.05
CV = chi2.ppf(1 - alpha, 2)  # 卡方分佈的臨界值，自由度為2

fig, ax = plt.subplots(2, 4, figsize=(20,10 ))  # Create a 2x3 grid of
subplots
ax = ax.flatten()  # Flatten the array of axes to make it easier to
iterate over
xx = np.linspace(0, 15, 200) # 卡方分佈的x 軸範圍

for i, n in enumerate(sample_sizes):
    x = norm.rvs(0, 1, size=(N, n))

    # 計算G_3 的值
    g1 = np.sqrt(n / 6) *skew(x,axis=1) # G_1
    g2 = np. sqrt(n/24)*kurtosis(x, bias=True, axis=1)  # G_2
    g3 = g1**2 + g2**2  # G_3 統計量

    # 畫出直方圖和卡方分佈曲線
    ax[i].hist(g3, bins=100, density=True, label='Jarque-Bera',
alpha=0.7, color='skyblue', edgecolor='black')
    ax[i].plot(xx, chi2.pdf(xx, 2), label='Chi-Squared (2)',
```

```
color='orange', linewidth=2) #  卡方分佈的PDF 進行比較

    reject = g3 > CV
    reject_rate = np.mean(reject)

    ax[i].set_title(f'n={n}, Rejection Rate={reject_rate:.4f}',
fontsize=12)
    ax[i].legend(fontsize=10)
    ax[i].set_xlim(0, 25)

plt.tight_layout()
plt.show()
```




```
from scipy.stats import norm, kurtosis, skew ,chi2
import numpy as np
import matplotlib.pyplot as plt

# 定義計算G_3 函數
def calculate_G3_vectorized(n):
    xs = norm.rvs(size=(N, n))  # 生成多組樣本，N 組，每組 n 個
    g1 = np.sqrt(n / 6) * skew(xs, axis=1)  # 偏度
    g2 = np.sqrt(n / 24) * kurtosis(xs, bias=False, axis=1)  # G_2
    g3 = g1**2 + g2**2  # G_3
    return g3

# 定義兩種不同的樣本大小
n_values = [10, 500]  # 樣本大小
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
N = 50000  # 實驗次數
xchi = np.linspace(0, 20, 1000)
```

```
for i, n in enumerate(n_values):#這裡利用enumerate 來列舉出sample_sizes 裡面的值i 沒有給定所以是從0 開始
    g3_values = calculate_G3_vectorized(n)  # 計算G_3 的值

    x_sorted = np.sort(g3_values)  # 排序G_3 值
    Y = np.arange(1, N + 1) / N
    row = i // 2  #這裡是用來決定圖形要放在哪一列i 帶入0 會得出0//2=0 代表放在第一列
    col = i % 2  #這裡是用來決定圖形要放在哪一行i 帶入0 會得出0%2=0 代表放在第一行
    ax = axs[row, col] #可以利用這種方式來編排圖形位置

    # 繪製ECDF 圖和卡方分布的CDF 進行比對
    ax.plot(x_sorted, Y, drawstyle='steps-pre', label='Empirical CDF')

    Y_chi= chi2.cdf(x_sorted,df=2) #
    ax.plot(x_sorted, Y_chi, color='r', linestyle='--', linewidth=2, alpha=0.5, label='Real CDF')
    ax.legend()
    ax.grid(True)
    ax.set_title('Empirical CDF with {} samples'.format(n), fontsize='small')

    # 畫出G_3 的直方圖和卡方分佈自由度2 的PDF 對比
    ax_hist_pdf = axs[1, col]
    ax_hist_pdf.hist(g3_values, color='lightblue', edgecolor='y', bins=50, density=True, alpha=0.7, label='G3 Histogram')
    x = np.linspace(0, 15, 1000)  # 調整x 的範圍以適應G_3 值的範圍
    pdf = chi2.pdf(x, 2)  # 卡方分佈的PDF
    ax_hist_pdf.plot(x, pdf, color='g', label='Chi-Squared PDF (2)')
    ax_hist_pdf.legend()
    ax_hist_pdf.set_title('G3 Histogram and Chi-Squared PDF with n = {}'.format(n), fontsize='small')
    ax.set_xlim(0, 4)

plt.tight_layout()
plt.show()
```



注意事項與討論:

- 在小樣本其實可以大概看出是卡方分布，但大樣本是更加明顯的。
- 在樣本數小的時候，reject rate 只到 0.0085 離 0.05 還有一段距離，樣本數越大的時候 reject rate 來到 0.05 附近。
- 從圖形角度來看，可以發現樣本數小的時候圖形左邊堆積過多拒絕的不夠多進而導致 reject rate 太小，而樣本數大的時候，不符合卡方分配(df=2)，可以看到圖形貼合卡方分配(df=2)，符合卡方分配。
- 一樣，在 cdf 圖也可以發現當樣本比較小時曲線的偏移量是大的，但當樣本數在 n=500 上升時就會幾乎貼近常態分佈的曲線。
- 這裡 bins 的數量可以調高一點 這樣圖的條數會更多，可讀性更高。

研究四:

將上述驗證程式改寫為一個副程式，假設取名為 stats, p_value = JB_test(x)，輸入參數 x 代表欲檢定是否為常態的一組資料。 輸出兩個結果，stats 為 G_3 檢定統計量的值，p_value 為檢定的 p-value。

```python
import numpy as np
from scipy.stats import norm, chi2, skew, kurtosis

# def jb_test
def JB_test(x):
    g1 = np.sqrt(n / 6) * skew(x, bias = False,axis=1)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias = False,axis=1)
    g3 = g1 ** 2 + g2 ** 2
    pvalue=chi2.sf(g3, df = 2)
    return g3, pvalue
```

注意事項與討論:

- 這裡定義一個 JB 的函數運用

研究五:

接著檢驗檢定統計量 G_3 的檢定力。採蒙地卡羅模擬方式，步驟如下：

- 從下列的分配母體中抽樣：$N(0,1), T(3), T(10), T(30), U(0,1), \chi^2(8)$。
- 抽樣數 $n=10,20,30,50,100,300,500$。
- 實驗次數 $N=50000$。
- 型一誤 $\alpha=0.05$。
- 對每個分配母體與樣本數，分別計算檢定力： $Power=P(Reject H_0 \vee H_a)$，其中 H_0: 資料來自常態；H_a: 資料來自其他分配。 最後針對每個母體，繪製如下圖的 Power vs. sample size。觀察檢定力受樣本數與母體來源（與常態的相似度）的影響。其中 Y 軸必須選擇合適的範圍，方能呈現出清楚的 power 值。
- 其實，當 H_a 來自常態時（也就是資料來自 H_0 的意思），此時的 Power 又稱為顯著水準，且理論上， Power 應該維持在所設定的型一誤 α，即 0.05，如下圖左。因此，當檢定統計量無法維持既定的顯著水準時，後續的檢定力也不用做了。因為檢定統計量是根據 H_0 為真的條件下得到的，連「自己的出身」都維持不住，則檢定力也失去意義。
- 一般而言，檢定力會隨著樣本數增加而變大，亦即，樣本數大有利於辨認資料的「真偽」

左方那張圖表示 : 當資料來自 H_0，檢定力也稱「顯著水準」，理論上必須維持在 α 值。 而右方那張圖表示 : 當資料來自 H_a，檢定力隨著樣本數增加而變大

- 上述的 JB 統計量或稱 JB test 可以拿來與其他常態檢定的產品比試一下。譬如，scipy.stats 也提供了 Jarque–Bera test(**jarque_bera**), D'Agostino and Pearson's (**normaltest**), Kolmogorov-Smirnov test(**kstest**), Shapiro-Wilk test(**shapiro**) 與 Anderson-Darling test(**anderson**) 等常態檢定的指令。下圖以 Anderson-Darling test 與本專題的 JB test 比較（蒙地卡羅實驗 5 萬次），左圖為顯著水準的維持，右圖為對 T(3) 分配的檢定力。從左圖看出 scipy 的 anderson 指令在小樣本時，型一誤的維持較差，即便樣本數升高也比 JB test 差。另，右圖在對 T(3) 的檢定力也比 JB test 差，直到樣本數上升到 300 才趕上（本實驗沒有嘗試樣本數介於 100~300 之間）。雖然從下圖的比較中，JB test 優於 scipy 的 anderson，但這不意味著 JB test 比 Anderson-Darling test 好，很有可能是 scipy 的這支 anderson 程式並沒有寫好。何況本專題的 JB test 在樣本數小於 2000 時，統計量並不滿足該有的卡方分配。讀者可以試試 scipy 的其他指令。

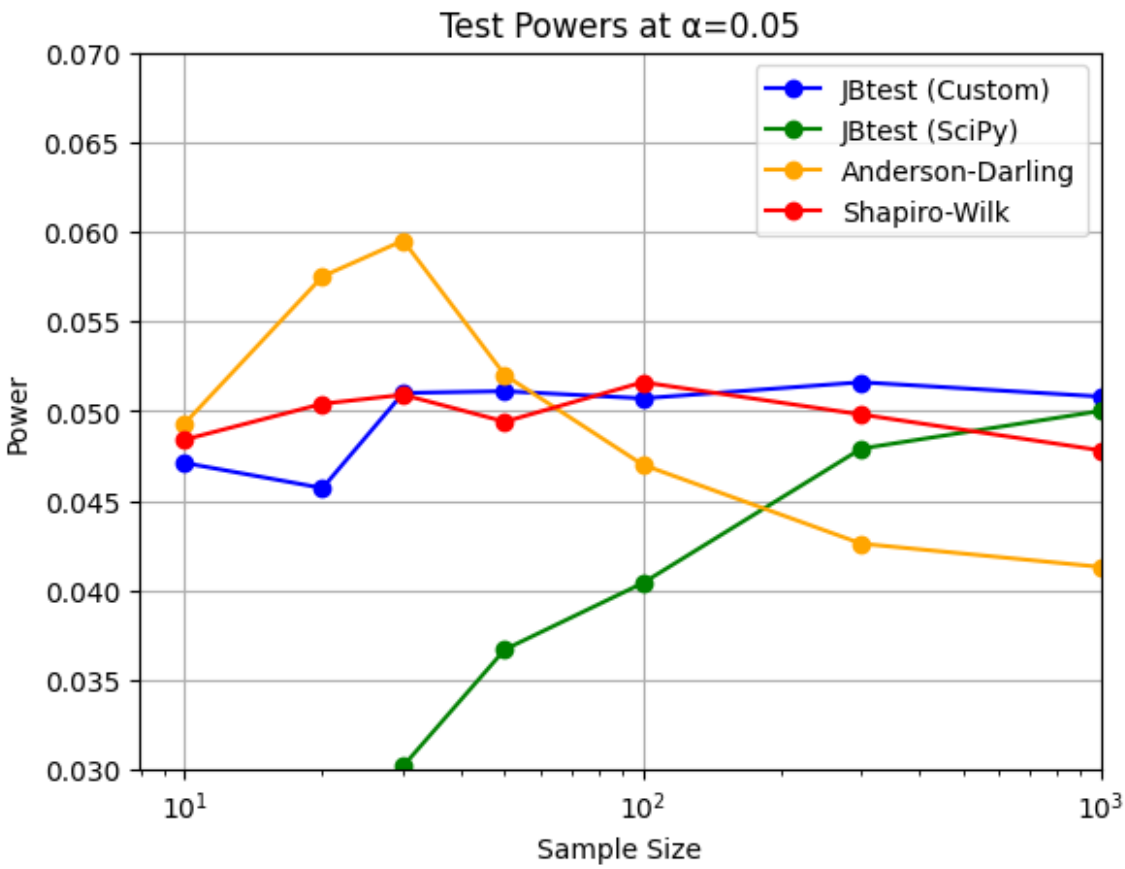
- scipy 的這幾個指令都只能對一組資料進行檢定統計量與 p-val 的計算，並不能 broadcasting 到 5 萬次。因此執行效率較本專題的 JB test 差很多。讀者可以細心的觀察兩者間執行時間的差異。

$N(0,1)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2
import matplotlib.pyplot as plt

# 定義JB_test 函數
# Define the JB_test function
def JB_test(x):
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列
dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果
為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的
jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結
果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = norm.rvs(0, 1, size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
```

```
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用
jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於
alpha

        # Shapiro-Wilk test
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0.03,0.07 )
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\1238529072.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 從圖中可以看到 scipy 的 jb_test 是慢慢爬上來的而不是一開始就在 0.05 附近。
- scipy 的 jb_test 和 anderson 以及自訂的 jb_test 檢定力，最後都會 0.05 附近浮動(將 ylim 設置在(0.03,0.07))
- 圖中可以發現樣本數小的時候，anderson 的 power 是高於其他兩個的，而後來自訂的 jb_test 在樣本數超過 100 後就追上了，scipy 的 jb_test 也在之後超越了 anderson 。
- 檢定力表現在樣本數為 1000 時的優劣排序為自訂的 jb_test>scipy 的 jb_test>sharpio>anderson
- anderson 的檢定力不只在最後最低，且樣本大小的改變會影響他的起伏許多，最不推薦使用。
- scipy 的 jb_test 檢定力則在樣本小的時候沒有檢定力，要到樣本數足夠才建議使用。
- 抽樣數在 50000 時跑不太出來，已 10000 替代。

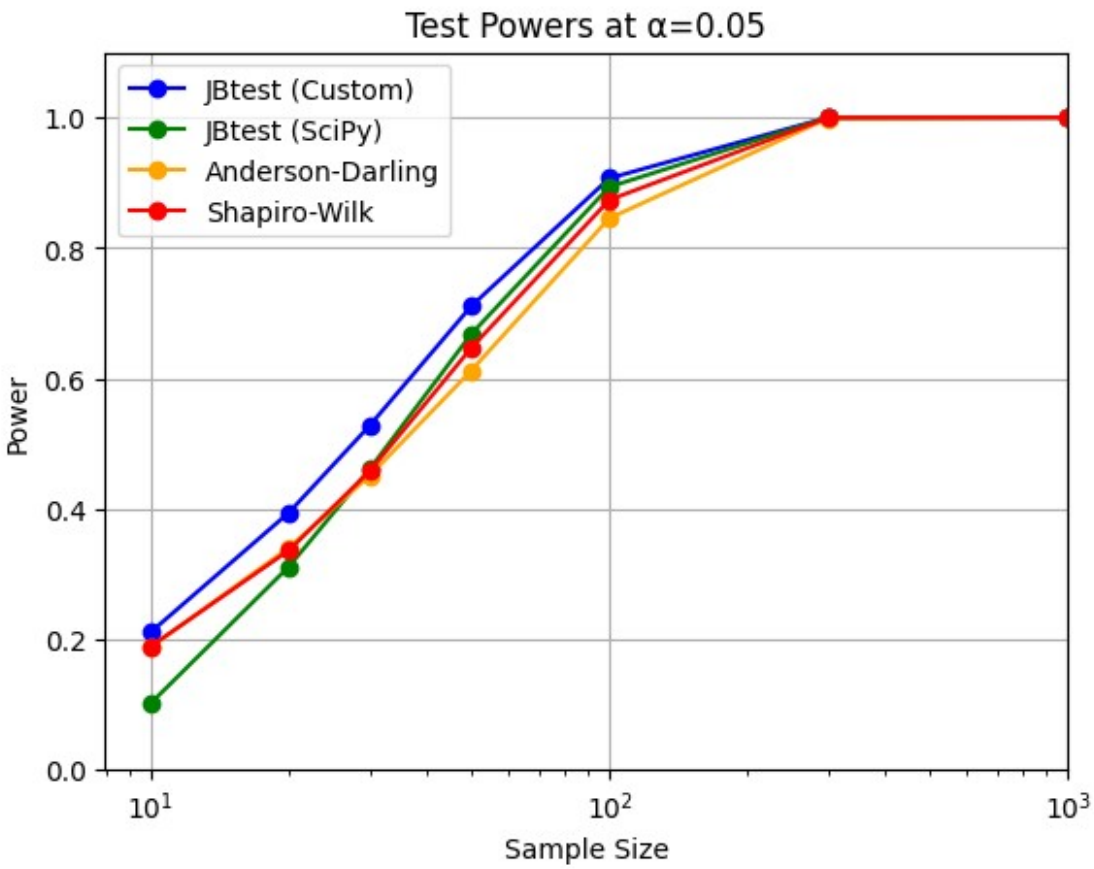
$T(3)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2
import matplotlib.pyplot as plt

# 定義JB_test 函數
```

```python
# Define the JB_test function
def JB_test(x):
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列
dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果
為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的
jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結
果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = t.rvs(3,0, 1, size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用
jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於
alpha

        # Shapiro-Wilk test
```

```
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0,1.1)
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\2326646268.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 圖片中可以看到 sharpio test、scipy 的 jb_test 和 anderson 以及自訂的 jb_test 檢定力都有相同往上爬的趨勢，彼此相差不多，自訂的 jb_test 可以說是比其他兩個好。
- 一開始 scipy 的 jb_test 有小的檢定力。
- sharpio 和 anderson 在樣本數小的時候有相同走向，但後來有了些許差距。
- 圖片中 Anderson 在樣本數 100 時已經被 scipy 的 jb_test 超過。
- sharpio test、scipy 的 jb_test 和 anderson 以及自訂的 jb_test 可以看到在樣本越來越大的表現會更好。
- 總體而言比較下來，優劣順序為 自訂的 jb_test>scipy 的 jb_test>sharpio test>anderson 但彼此相差不大。

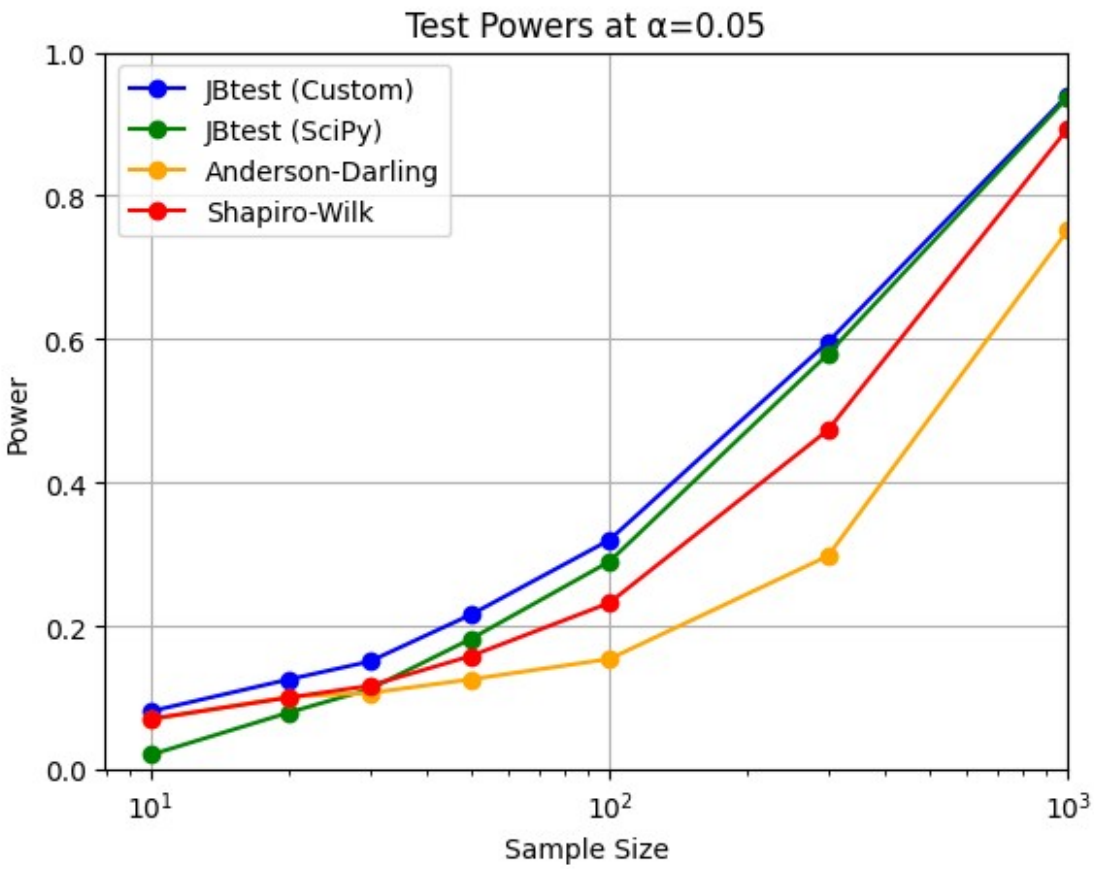
$T(10)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2
import matplotlib.pyplot as plt

# 定義JB_test 函數
# Define the JB_test function
def JB_test(x):
```

```
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列
dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果
為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的
jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結
果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = t.rvs(10,0, 1, size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用
jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於
alpha

        # Shapiro-Wilk test
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
```

```
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0,1)
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\313743297.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 圖片中可以看到 sharpio test、scipy 的 jb_test 和 anderson 以及自訂的 jb_test 檢定力都有相同往上爬的趨勢，彼此相差不多，自訂的 jb_test 依舊是比其他三個好。
- 圖片中可以看到 sharpio test、scipy 的 jb_test 和 anderson 以及自訂的 jb_test 檢定力在數值方面都是偏低的，到樣本數 100 時甚至都只有 0.3(相比前面的 0.9)
- 圖片中 Anderson 在樣本數 100 時已經被 scipy 的 jb_test 超過。
- scipy 的 jb_test 和 anderson 以及自訂的 jb_test 可以看到在樣本越來越大的表現會更好。
- scipy 的 jb_test 和 anderson 可以發現在樣本數 100 時檢定力差距變大了。
- 優劣順序為自訂的 jb_test>scipy 的 jb_test>sharpio test>anderson。
- anderson 在這種狀況下檢定力表現與其他相差更大，推測其他的更適合在此條件作檢定。

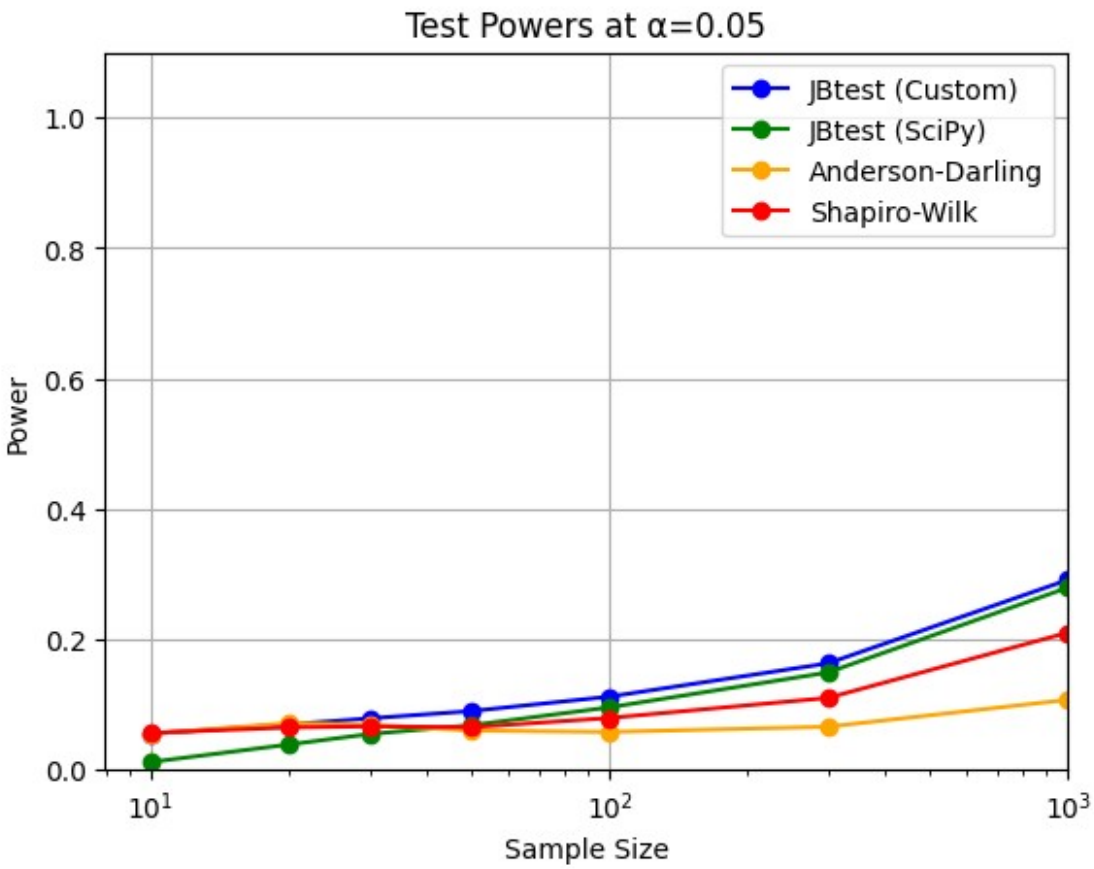
$T(30)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2
import matplotlib.pyplot as plt

# 定義JB_test 函數
# Define the JB_test function
```

```python
def JB_test(x):
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列 dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = t.rvs(30,0, 1, size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於alpha

        # Shapiro-Wilk test
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
```

```
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0,1.1)
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\3591496109.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 圖片中樣本數小的時候，檢定力只有 scipy 的 jbtest 會低一點，到 50 樣本數時就大概差不多了。
- 圖片中 scipy 的 jbtest 和自訂的 jbtest 兩者的趨勢相當，但是自訂的 jbtest 一開始檢定力從 0.05 左右開始上漲而 scipy 的 jbtest 從 0 開始。
- 圖片中 anderson test，在一開始樣本較小的時候有與自訂的 jbtest 相同走向，但樣本數稍大後，則平緩沒有上升，到了樣本超過 100 後才開始有了往上攀爬的一點趨勢。
- 圖片中 scipy 的 jbtest，和自訂的 jbtest，在樣本數到達 1000 時檢定力來到約 0.3，而 anderson 大概只有 0.01，sharpio test 則有 0.2 左右。
- 圖片中 優劣順序為自訂的 jb_test>scipy 的 jb_test>sharpio test>anderson。
- 會推測說 jb_test、scipy 的 jb_test 是最適合此狀況的。

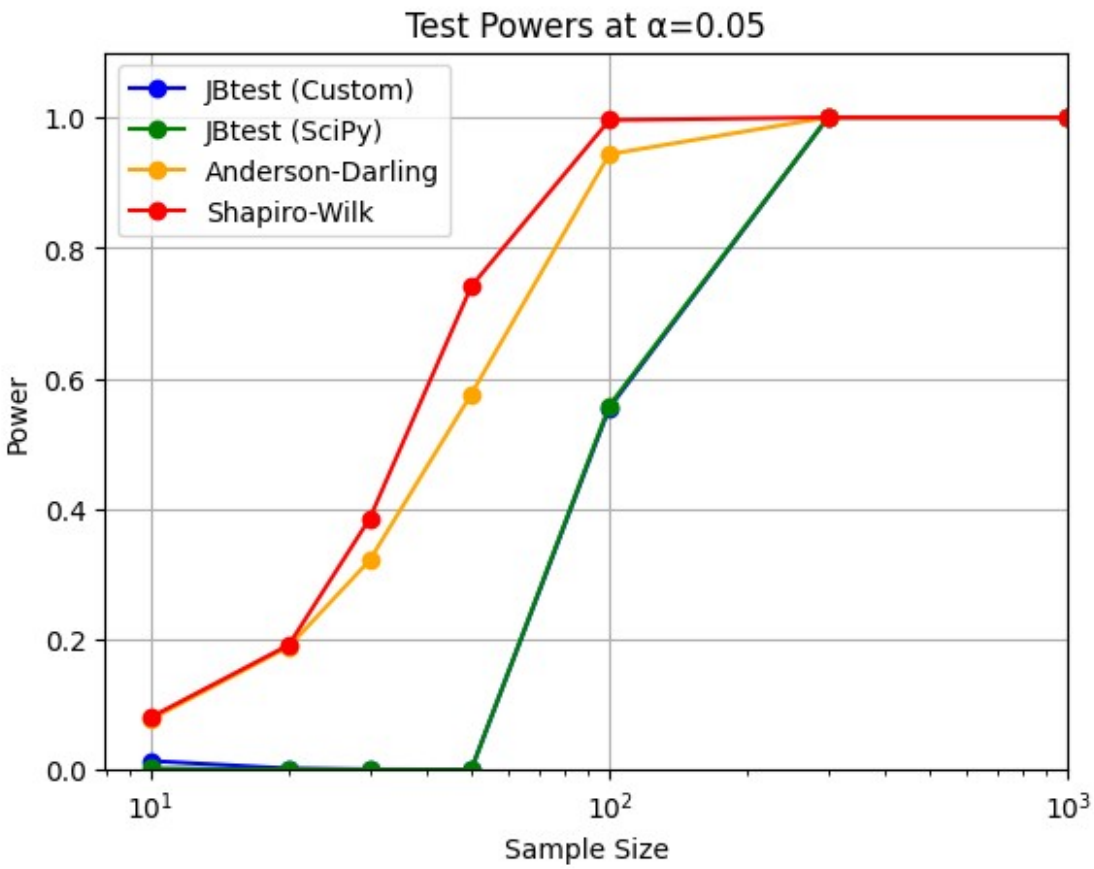
$U(0,1)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2,uniform
import matplotlib.pyplot as plt

# 定義JB_test 函數
# Define the JB_test function
```

```python
def JB_test(x):
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列 dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = uniform.rvs(0, 1, size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於alpha

        # Shapiro-Wilk test
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
```

```
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0,1.1)
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\3860113127.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 圖片中 scipy 的 jbtest，和自訂的 jbtest 兩者幾乎重疊，兩者之從 0 開始往上攀爬。
- 圖片中 anderson 和 sharpio，一開始檢定力不為 0，且有往上攀爬的趨勢。
- 在樣本數為 100 時，anderson 的檢定力來到 0.9，而 scipy 的 jbtest，和自訂的 jbtest 則位於 0.55 左右，明顯 anderson 較佳。
- 但是當樣本數更大(500)時，四者都到達 1。
- 推測 scipy 的 jbtest，和自訂的 jbtest 在這個狀況下對樣本數過小會有偏誤的發生，因此不適用。
- 推測適合使用 anderson 和 sharpio wilk 的方法，樣本數不用到特別大，就有不錯的檢定力。

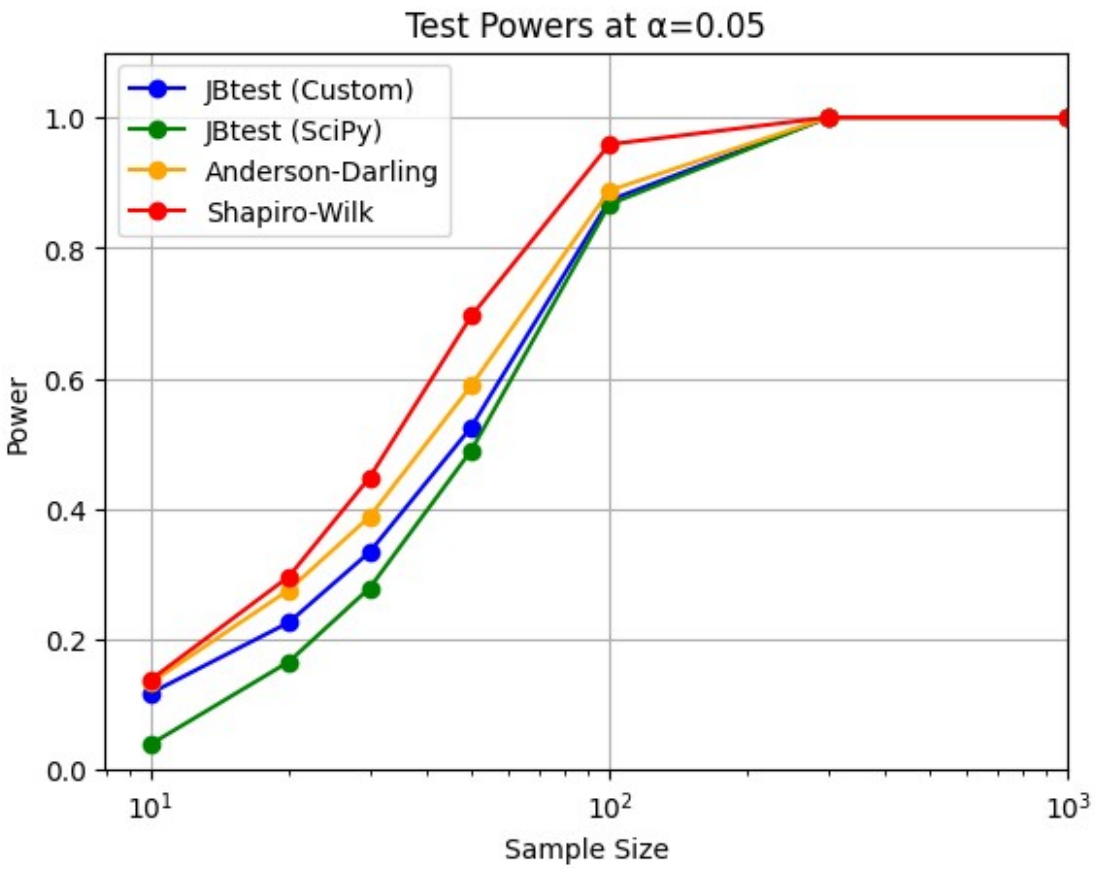
$\chi^2(8)$

```
import numpy as np
from scipy.stats import norm, anderson, t, jarque_bera,
shapiro ,skew,kurtosis,chi2,uniform
import matplotlib.pyplot as plt

# 定義JB_test 函數
# Define the JB_test function
def JB_test(x):
```

```
    n = len(x)
    g1 = np.sqrt(n / 6) * skew(x, bias=False)
    g2 = np.sqrt(n / 24) * kurtosis(x, bias=False)
    g3 = g1 ** 2 + g2 ** 2
    pvalue = chi2.sf(g3, df=2)
    return g3, pvalue

# 初始化變數

# 初始化列表
N = 10000
alpha = 0.05
sample_sizes = [10, 20, 30, 50, 100, 300,1000]
jb_powers = [] # 新增JB 檢定的列表
ad_powers = [] # 新增Anderson-Darling 檢定的列表
jb_powers_scipy = []  # 新增使用scipy 的jarque_bera 函數的列表
shapiro_powers = []  # 新增Shapiro-Wilk 檢定的列表

# 執行每個樣本大小的檢定
for sample in sample_sizes:
    jb_power = np.zeros(N, dtype=bool) # 初始化JB 檢定結果為False 的陣列
dtype=bool 代表陣列中的元素為布林值 是為了要計算拒絕率
    ad_power = np.zeros(N, dtype=bool) # 初始化Anderson-Darling 檢定結果
為False 的陣列
    jb_power_scipy = np.zeros(N, dtype=bool) # 初始化使用scipy 的
jarque_bera 函數的結果為False 的陣列
    shapiro_power = np.zeros(N, dtype=bool)  # 初始化Shapiro-Wilk 檢定結
果為False 的陣列
    #先進行不同樣本大小的抽樣
    for i in range(N):  #這裡的迴圈再把每個樣本大小 做N 次抽樣
        data = chi2.rvs(8,0,1,size=sample)

        # Custom JB test
        _, p_val_jb = JB_test(data) #這裡的_是因為我們只需要p-value
        jb_power[i] = p_val_jb < alpha # 記錄p-value 是否小於alpha

        # Anderson-Darling test
        result = anderson(data)  # 使用anderson 函數
        ad_power[i] = result.statistic > result.critical_values[2]

        # Using scipy's jarque_bera test
        jb_statistic_scipy, p_val_scipy = jarque_bera(data) # 使用
jarque_bera 函數
        jb_power_scipy[i] = p_val_scipy < alpha # 記錄p-value 是否小於
alpha

        # Shapiro-Wilk test
        shapiro_stat, p_val_shapiro = shapiro(data) # 使用shapiro 函數
        shapiro_power[i] = p_val_shapiro < alpha  # 記錄p-value 是否小於
```

```
alpha

    # 計算平均值並將其附加到列表中
    jb_powers.append(np.mean(jb_power))
    ad_powers.append(np.mean(ad_power))
    jb_powers_scipy.append(np.mean(jb_power_scipy))
    shapiro_powers.append(np.mean(shapiro_power))

# 畫出結果圖表
plt.plot(sample_sizes, jb_powers, marker='o', color='blue',
label='JBtest (Custom)')
plt.plot(sample_sizes, jb_powers_scipy, marker='o', color='green',
label='JBtest (SciPy)')
plt.plot(sample_sizes, ad_powers, marker='o', color='orange',
label='Anderson-Darling')
plt.plot(sample_sizes, shapiro_powers, marker='o', color='red',
label='Shapiro-Wilk')
plt.xticks(np.arange(len(sample_sizes)), sample_sizes)
plt.xscale('log')
plt.title('Test Powers at α=0.05')
plt.xlabel('Sample Size')
plt.ylabel('Power')
plt.ylim(0,1.1)
plt.xlim(0, 1000)
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Users\黃\AppData\Local\Temp\ipykernel_33320\623481321.py:69:
UserWarning: Attempt to set non-positive xlim on a log-scaled axis
will be ignored.
  plt.xlim(0, 1000)
```



注意事項與討論:

- 圖片中四者有差不多的走向。
- 圖片中 scipy 的 jbtest，自訂的 jbtest anderson 的趨勢幾乎一樣。
- 圖片中檢定力高低排序為 sharpio test>anderson >自訂的 jbtest >scipy 的 jbtest ，到樣本數約為 500 時檢定力到達了 1。
- 四種差距不大，感覺都是適用的，要說最好的是 sharpio test。在樣本數為 100 時多出了約 0.1 的檢定力。